

LASAL

Motion Control API 기능 설명서

LasalMotionControlLib 공개 C# API의
기능, 호출 인자 UNIT과 반환값

문서	API 기능 설명서
적용 API	LasalMotionControlLib 0.9.1-preview
환경	Windows, .NET Framework 4.8
발행	2026-08-20
문서 버전	2.4-development

API 기능과 호출에 필요한 값만 간단히 설명합니다.

개정 이력

문서 버전	날짜	내용
1.0	2026-07-15	최초 작성
1.1	2026-07-16	공개 API 레퍼런스 작성
1.2	2026-07-16	API 기능, 인자 UNIT과 반환값 중심으로 간소화
1.3	2026-07-16	preview 안전 경고, 응답 판정과 4축 group 제한 보완
1.4	2026-07-16	group position read 계약 불일치와 static identity 제한 명시
1.5	2026-07-22	read-only Admin, typed drive status, PI/Bulk facade와 local error catalog 추가
1.6	2026-07-23	0x7D22 GroupMoveLinearRelative API, wire/state 제한과 runtime 검증 경계 추가
1.7	2026-07-29	Axis Power/Reset/Stop과 Group Stop accepted-once stable wait, deadline evidence 추가
1.8	2026-07-29	Axis Stop/Reset process-local mutation 귀속, Reset Begin/Resume와 WPF no-replay 정책 추가
1.9	2026-07-30	신규 wait/resume, drive error read와 D0~D5/Recorder/Topology API를 반영하고 current 지원 상태와 완료 판정 경계 추가
2.0-candidate	2026-07-31	Axis1-only SDO Write identity-pinned four-ticket gate, stale recovery retirement, single-instance 실행과 transactional Distribution candidate 경계 추가
2.1-candidate	2026-08-04	LMC Home current-position-zero start/outcome/retirement, DS402 Home gate 상태와 TW19/TW20 encoder maintenance 계약 추가
2.2-candidate	2026-08-11	14ccf58 exact canonical -1 bounded fresh-TCP reconnect, complete local cleanup, startup identity와 PC 검증 경계 추가
2.3-candidate	2026-08-12	cbf2548 actual EXE X 종료/재실행과 binary identity gate, 3c63dea 13-role active Python dependency closure, d4204b4 exact Gate D PC/static snapshot 승인, RPC_INIT_FRESH_TCP_ONCE_V2 bounded pre-response transport recovery와 canonical tracked release-input 경계 추가
2.4-development	2026-08-20	current API 문서 위치 통합, SetPosition P1/volatile backing/fail-closed 계약과 최신 PLC image load 경계 반영

이 문서는 LasalMotionControlLib.dll의 API 기능, 호출 인자, UNIT, 반환값과 안전 제약을 설명하는 current 기준 문서다. 구현률, 시험 수치, artifact identity와 다음 작업은 API 개발 진척도에서만 관리한다. byte offset과 frame shape의 정보는 코드와 함께 검증되는 DINT packet map이다.

Preview/안전 경고: 0.9.1-preview는 production 승인본이 아니다. 최신 build/download와 기능별 검증 상태는 API 개발 진척도에서 확인한다. LMC_Response.IsSuccess는 frame과 command 수락 결과이지 motion, power 전이 또는 Stop 완료가 아니다. typed status/position을 polling하고 stable sample과 final readback을 별도로 확인한다. CloseConnection, Dispose, timeout과 cancellation은 PLC motion Stop이나 safe-stop을 보내지 않는다. 실제 장비에서는 E-stop, HW/SW limit, UNIT, Home/Reference와 이동 범위를 별도 승인해야 한다.

SetPosition 경고: 0x7D12/0x7D14/0x7D1A의 C# API와 LASAL P1 source는 존재하지만 current PLC는 Store와 ordinary ownership이 FALSE, max-jump가 0, capability bits 3/5/70이 OFF이고 Admin SetPosition native call이 0회다. 1344-byte backing은 ordinary VAR_GLOBAL이며 restart/power-loss durability가 없다. 따라서 current image에서 SetPosition success, restart replay, durable query 또는 retirement를 사용하면 안 된다. exact request는 storage unavailable/detail 24로 fail-closed되는 것이 정상이다.

목차

개정 이력	2
1. 공통 사항	5
1.1 Assembly	5
1.2 UNIT 변환	5
1.3 공통 반환값	5
1.4 Enum 값	6
1.5 현재 지원 상태와 완료 판정	6
1.6 accepted-once wait/resume 공통 모델	7
1.7 WPF recovery identity mismatch 해소	7
2. Connection API	8
2.1 LMCCConnection	8
2.2 LMCCConnectionOptions	8
2.3 RplInitConnection	8
2.4 CloseConnection	10
2.5 Safety transport preemption	10
3. Single Axis API	11
3.1 LMCSingleAxis 생성	11
3.2 PowerOn	11
3.3 PowerOff	12
3.4 Reset	12
3.5 Stop	13
3.6 ReadStatus	15
3.7 GetActualPosition	15
3.8 MoveAbsoluteEx	15
3.9 MoveRelativeEx	16
3.10 MoveVelocityEx	17
3.11 Drive operation mode와 composite status	17
3.11.1 SetOperationMode 개발 SDK	18
3.12 Move 완료와 restart recovery 경계	19
4. Group API	19
4.1 LMCGroupAxis 생성	19
4.2 GetGroupMembersInfo	19
4.3 GroupPowerOn	20
4.4 GroupPowerOff	20

4.5 GroupEnable	21
4.6 GroupDisable	22
4.7 GroupReset	22
4.8 GroupStop	24
4.9 GroupReadStatus	25
4.10 GroupReadActualPosition	25
4.11 SetKinTransformCartesian4Axis	25
4.12 MoveLinearAbsoluteEx	26
4.13 MoveLinearRelativeEx	27
4.14 LMCGroupMotionOptions	28
5. Return Type	28
5.1 LMCReadStatusResult	28
5.2 LMCReadActualPositionResult	29
5.3 LMCGroupReadStatusResult	29
5.4 LMCGroupReadActualPositionResult	30
5.5 LMCGroupMembersInfoResult	30
5.6 LMCAdminResponse	30
6. Admin과 Diagnostics facade	31
6.1 Admin capability와 semantic parameter read	31
6.2 PI alias와 Bulk builder/reader	32
6.3 Project-local error catalog	32
6.4 Diagnostics D0 capability와 공통 수명	32
6.5 Diagnostics D1 Catalog, Health와 PI Read	33
6.6 Diagnostics D2 Bulk Read	33
6.7 Diagnostics D3/D4 Recorder	34
6.8 Diagnostics D5 SDO Read와 operation ticket	34
6.9 Static Topology와 Dynamic Node/I/O	34
6.10 Mutation API 정책	35
6.11 Request/result와 provenance 확인	36
6.12 LMC Home, DS402 Home과 Encoder Maintenance	36
LMC Home CurrentPositionZero	36
DS402 Home method 37	37
TW[20] / TW[19] Encoder Maintenance	37
6.13 Axis SetPosition	37

1. 공통 사항

1.1 Assembly

항목	값
DLL	LasalMotionControlLib.dll
Namespace	LasalMotionControlLib
Framework	.NET Framework 4.8
API version	0.9.1-preview

1.2 UNIT 변환

API의 motion 인자는 `Int32` DINT다. API는 UNIT을 자동으로 곱하거나 나누지 않는다.

송신 DINT = 물리값 x PLC application UNIT
 수신 물리값 = 수신 DINT / PLC application UNIT
 Jerk 송신 DINT = (물리 jerk / 1000) x PLC application UNIT

값	API 인자 UNIT	설명
Position / Distance	PLC application UNIT DINT	예: mm UNIT이 10000이면 1 mm는 10000
Velocity	PLC application UNIT/s DINT	예: 1 mm/s는 UNIT 10000 기준 10000
Acceleration / Deceleration	PLC application UNIT/s ² DINT	PLC에 설정된 application UNIT 사용
Jerk	PLC application UNIT/s ³ /1000 DINT	1000 mm/s ³ 는 입력값 1에 UNIT을 곱함
Actual position	PLC application UNIT DINT	반환값을 UNIT으로 나누어 물리값 계산

주요 상수는 다음과 같다. 실제 PLC에 설정된 UNIT이 다르면 그 값을 사용한다.

Constant	값	의미
LMC_Units.MM	10000	Millimeter
LMC_Units.M	10000000	Meter
LMC_Units.DEG	10000	Degree
LMC_Units.MMPSEC	10000	Millimeter/second
LMC_Units.RPM	1000	Revolution/minute

1.3 공통 반환값

제어와 motion 명령은 `LMC_Response`를 반환한다.

Property	Type	설명
IsSuccess	bool	frame과 command 결과 성공 여부; motion/power/stop 완료를 뜻하지 않음
Raw / Payload	byte[]	방어 복사된 원본 frame/payload
HeaderStatus	ushort	response envelope 상태
PayloadLength	ushort	header에 선언된 payload 길이

Property	Type	설명
HeaderReserved	uint	header reserved 값
IsFrameValid	bool	header와 command별 payload shape 검증 결과
HasCommandResult	bool	command status/error field 존재 여부
CommandStatus	ushort	command/function status
Status	ushort	command result가 있으면 CommandStatus, 없으면 HeaderStatus
ErrorId	short	반환된 error ID, 정상은 0

비동기 메서드는 동일한 결과를 Task<LMC_Response>로 반환한다.

1.4 Enum 값

Enum	사용 값	설명
LMC_DIRECTION	Shortest	Absolute / Relative motion
LMC_DIRECTION	Positive, Negative	Velocity motion 방향
LMC_COORD_SYSTEM	None, Acs, Mcs, Pcs	Group 좌표계
LMC_BUFFER_MODE	Aborting, Buffered	현재 배포 PLC에서 사용하는 buffer mode
LMC_GROUP_TRANSITION_MODE	ExactStop, ContinuousDirect	현재 배포 PLC에서 사용하는 transition mode

1.5 현재 지원 상태와 완료 판정

아래의 source-active는 C# API와 LASAL route가 current source에 있다는 뜻이다. production 승인이나 전체 실제 장비 검증을 뜻하지 않는다.

영역	Current 상태	사용 범위와 남은 확인
Connection / Axis / Group core	source-active, PLC 검증 부분	대표 lookup, power, move, stop/read 경로가 있으나 전체 command/fault/race matrix는 미완료
Axis/Group accepted-once wait/resume	PC public API, 기존 opcode 재사용	ACK 뒤 status-only polling과 no-replay recovery를 제공하며 current same-hash/실장비 전체 회귀가 필요
Admin 0x7D00/10/20/22	source-active, 대표 PLC 검증	semantic read와 Group relative move의 전체 축/UNIT/fault matrix는 미완료
Axis SetPosition 0x7D12/14/1A	C#/LASAL P1 source, runtime fail-closed	Store/ownership OFF, volatile backing, bits 3/5/7 OFF, max-jump 0, native call 0; current 지원 API로 사용 금지
LMC Home 0x7D13/18/19	source-active, Admin bit 4 ON	CurrentPositionZero는 no-motion/no-switch이며 application position reset과 실제 encoder/multi-turn 효과를 구분해야 함
DS402 Home 0x7D15/16/17	method 37 source 구현, Admin bit 6/gate OFF	current runtime 실행 금지; source 존재와 terminal proof를 구분
TW[20]/TW[19] 0x7E53/54/55	fixed 0x20FC:2/:1 <- UInt16 1, source bits 18/19 ON	protocol terminal과 선택 drive의 실제 물리 효과를 별도로 검증
D1/D2/Recorder/D5 Read	C# facade와 PLC route가 단계별 구현	capability를 먼저 읽고 fault, reconnect, soak와 physical readback evidence를 추가해야 함
Static EtherCAT topology 0x7E11/12	configured 7-entry inventory	topology qualifier durable report와 current PLC identity 확인 필요
Node Health / Digital I/O 0x7E13/22/23	0x7E13/22 LASAL source 구현, capability OFF; 0x7E23 없음	capability activation과 runtime/hardware proof 전에는 정상 UI/API에서 호출하지 않음

영역	Current 상태	사용 범위와 남은 확인
SDO Write	축 1 UI[24] exact target만 source/IDE build 승인	bit 9와 exact identity를 확인한 제한 시험만 허용; current 실기 mutation evidence 미완료
PI Write / Recorder Double	gate 또는 allowlist OFF	별도 승인과 실제 장비 mutation evidence 전까지 사용하지 않음

완료 판정은 ACK -> typed status polling -> stable sample -> final readback 순서로 한다.

timeout/cancellation/connection close는 장비 정지나 command 취소를 자동 보장하지 않는다.

1.6 accepted-once wait/resume 공통 모델

Power, Stop, Reset과 일부 Group command는 ACK와 완료 상태를 분리하고, 한 번 수락된 command를 다시 보내지 않은 채 status-only polling을 재개하는 API를 제공한다.

구성	기본값 / 의미
...WaitOptions.TotalTimeoutMilliseconds	5000 ms, 허용 1~600000 ms
...WaitOptions.PollIntervalMilliseconds	50 ms
...WaitOptions.StableSampleCount	3회, 허용 1~100회
...SubmissionOutcome	NotAttempted, Rejected, OutcomeUncertain, Accepted
...WaitContinuation	같은 connection/session/object에 묶인 pending ACK와 polling evidence
...WaitResult	final status, continuation과 submission/polling evidence
Pending...WaitContinuation	현재 handle에서 아직 완료되지 않은 latest accepted continuation; 없으면 null

```
LMCAxisStopWaitContinuation pending =
    await axis.BeginStopWaitForStableStandstillAsync(
        deceleration,
        jerk,
        cancellationToken);

LMCAxisStopWaitResult stopped =
    await axis.ResumeStopWaitForStableStandstillAsync(
        pending,
        cancellationToken);
```

Continuation은 server-side idempotency key나 영구 token이 아니라 현재 process의 connection/session-bound 객체다. exact pending continuation으로 Resume...을 호출해야 하며, 새 Begin... 호출을 blind retry로 사용하지 않는다. pre-write cancellation은 보통 zero-wire지만, CommandMayHaveBeenSent=true 또는 OutcomeUncertain이면 자동 replay하지 않는다. TransportInvalidatedAtDeadline=true이면 reconnect와 object lookup을 다시 수행한다. pending property는 durable storage가 아니므로 process restart 뒤 복원되지 않는다. restart 뒤에는 motion command를 재전송하지 말고 read-only WaitFor... helper 또는 explicit Stop/Power Off를 사용해 현재 상태를 다시 확정한다. 단, API가 별도 durable recovery record/attach 계약을 명시한 Axis/Group operation은 old continuation을 복원하는 대신 exact endpoint/PLC/target identity를 재검증해 새 session-bound status-only continuation을 만든다. 이 경로도 원 command를 replay하지 않는다.

1.7 WPF recovery identity mismatch 해소

개발 WPF가 저장된 recovery record와 현재 PLC의 BootId 또는 MapRevision 불일치를 표시하면 Motion, Power, Group control, D5/SDO Write와 qualification은 모두 차단된다. 이 상태에서 현재 PLC의 status가 정상이라는 이유로 과거 command 결과를 성공으로 판정하면 안 된다. Group Reset record는 DiagnosticsBuild도 exact identity에 포함하므로 Build-only mismatch도 같은 read-only quarantine 대상이다.

해당 record가 현재 endpoint의 이전 PLC identity에 속하고 운영자가 장비와 드라이브의 물리 상태를 독립 확인한 경우에만 화면의 stale recovery 목록을 검토한다. 확인 checkbox와 경고창을 승인한 뒤 Archive and Retire Stale Recovery를 실행한다. 이 작업은 PLC command를 보내지 않고 원 journal과 SHA-256을 immutable retirement ledger

에 보존한 뒤 exact journal CAS로만 record를 retire한다. 혼합 record에서는 현재 endpoint의 stale subset만 **RETIRE STALE**로 폐기하고 exact-current와 다른 endpoint record는 각각 **KEEP EXACT CURRENT**, **KEEP OTHER ENDPOINT**로 보존한다.

완료되면 기존 연결이 닫히고 앱이 종료된다. 새 process에서 다시 연결해 남은 exact status-only recovery를 완료해 야 Motion/Power/approved SDO Write admission이 재평가된다. ledger 또는 journal fault, endpoint 불일치, 실행 중 operation이 있으면 절차는 fail-closed다. retirement는 Build-bearing Group Reset record에 대해서는 current DiagnosticsBuild까지 confirmation 전후 두 번 다시 읽고, 기존 Build-less record는 이전 BootId/MapRevision 계약을 유지한다.

2. Connection API

2.1 LMConnection

Connection 객체를 생성한다. 생성만으로 PLC에 연결되지는 않는다.

<pre>public LMConnection() public LMConnection(LMConnectionOptions options)</pre>			
Parameter	Type	UNIT	설명
options	LMConnectionOptions	-	Connection timeout 설정
Return		설명	
LMConnection		생성된 connection 객체	

2.2 LMConnectionOptions

Connection timeout과 callback 검증 값을 설정한다.

Property	Type	UNIT	Default
ConnectTimeoutMilliseconds	int	ms	3000
ReceiveTimeoutMilliseconds	int	ms	3000
SendTimeoutMilliseconds	int	ms	3000
CallbackThreadJoinTimeoutMilliseconds	int	ms	500
ValidateCallbackSourceAddress	bool	-	true
CallbackRegistrationMode	LMCallbackRegistrationMode	-	LegacyRaw
CallbackRequestedMaxDatagramBytes	int	bytes	512
SendPriorityCoordinator	LMCSendPriorityCoordinator	-	선택적 PC-side safety send coordination

2.3 RpcInitConnection

PLC TCP 연결, RPC 초기화와 callback 등록을 수행한다.

```

public void RpcInitConnection(
    string remoteAddress,
    int remotePort,
    string localAddress)

public void RpcInitConnection(
    string remoteAddress,
    int remotePort,
    string localAddress,
    int callbackPort,
    uint eventMask)

public Task RpcInitConnectionAsync(
    string remoteAddress,
    int remotePort,
    string localAddress,
    CancellationToken cancellationToken)

public Task RpcInitConnectionAsync(
    string remoteAddress,
    int remotePort,
    string localAddress,
    int callbackPort,
    uint eventMask,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT / 범위	설명
remoteAddress	string	IPv4	PLC IP address
remotePort	int	1~65535	PLC TCP port
localAddress	string	IPv4	PLC와 연결되는 PC NIC address
callbackPort	int	0~65535	PC UDP callback port, 기본값 5003
eventMask	uint	Bit mask	Callback event mask, 기본값 0xFFFFFFFF
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
void	동기 초기화 완료
Task	비동기 초기화 작업

library default `LegacyRaw`는 callback registration 12-byte request/4-byte ACK를 사용한다. 명시적 `Version2WakeHint`는 32/20과 strict 52-byte typed wake를 사용한다. 이 mode에서 outer `Status=1`, `HeaderReserved=0`, payload 4, command `Status=1`, `ErrorId=-1`인 exact canonical init failure만 20 ms 뒤 같은 TCP socket에서 `0x8080`을 한 번 더 보낸다. `LastRpcSessionInitializationEvidence`는 `Outcome`, `AttemptCount`, `CanonicalRetryUsed`, 첫/마지막 response를 cleanup 뒤에도 보존한다. legacy, `ErrorId=0`/다른 error, nonzero reserved와 malformed response는 SDK retry가 없다.

SDK 자체는 새 TCP connection을 자동으로 만들지 않는다. 개발 WPF의 current `RPC_INIT_FRESH_TCP_ONCE_V2`는 첫 candidate에만 fresh-TCP budget을 준다. (A) exact canonical -1 두 개로 `AttemptCount=2`, `CanonicalRetryUsed=true`가 된 persistent same-socket failure는 100 ms 뒤, (B) 실제 `0x8080` request가 시작된 `AttemptCount=1`, response 없음과 direct `EndOfStreamException/SocketException/TimeoutException`, 또는 그중 하나를 `InnerException` chain에 가진 `IOException`인 pre-response transport failure는 1000 ms 뒤 새 `LMCConnection/TCP` 하나를 연다.

두 번째 candidate failure는 terminal이다. Connect-before-init은 `AttemptCount=0`이며 retry가 없다. cancellation, `ObjectDisposedException`, `InvalidDataException`(허용형 `InnerException`이 있어도 포함), malformed response, valid non--1 response, response 이후와 callback-stage failure에도 fresh-TCP retry가 없다. One UI operation is bounded to TCP 2 and `0x8080` 4 requests. `0x405C`는 init 성공 뒤에만 나가며 정상 registration ACK까지 받아야 Connect가 성공한다. Historical `14ccf58` V1은 persistent--1만 허용했던 과거 policy다.

100/1000 ms는 PC bounded backoff이지 PLC readiness 증거가 아니다. canonical wire -1만으로 내부 disarm -8/-9와 다른 lifecycle/ownership rejection을 구분할 수 없다. PC fake-RPC와 loopback test는 PLC runtime proof가 아니다. PC cleanup은 PLC disarm 성공을 뜻하지 않으며 private PLC state를 force-clear하지 않는다. Same-window Close -> Connect live reconnect is not verified.

2.4 CloseConnection

RPC connection과 local TCP/UDP resource를 닫는다.

```
public void CloseConnection()
public Task CloseConnectionAsync(Cancellation_token cancellationToken)
public void Dispose()
```

Parameter	Type	UNIT	설명
cancellationToken	Cancellation_token	-	비동기 호출 취소 token

Return	설명
void	동기 종료 완료
Task	비동기 종료 작업

CloseConnection, CloseConnectionAsync와 Dispose는 Axis/Group Stop 또는 Power Off를 보내지 않는다. strict CloseConnection[Async]는 초기화된 연결에서 가능한 경우 RPC close 0x405D를 시도한 뒤 local socket/resource를 정리한다. Dispose는 캡처한 close protocol/transport 오류를 local cleanup 경로 뒤 다시 throw하지 않지만, failed/uninitialized candidate에서는 0x405D가 나가지 않을 수 있다. nonzero close ACK나 close transport 오류가 있어도 local cleanup 뒤 RpcCloseResponse와 LastCloseException을 확인할 수 있다. strict CloseConnection[Async]는 cleanup 뒤에도 오류를 호출자에게 전달한다. 장비를 안전하게 정지해야 하면 connection close 전에 use an explicit safe-stop procedure를 적용한다.

개발 WPF의 내부 replacement와 창 X는 공용 최대 2회 Dispose cleanup 후 Disconnected, TCP/RPC/callback 정지와 endpoint null을 모두 요구한다. X는 이 postcondition이 완성되지 않으면 종료를 취소한다. startup log에는 ReconnectPolicy=RPC_INIT_FRESH_TCP_ONCE_V2, SdkPath, SdkBuildUtc가 있으며 topology marker V5는 유지된다.

RPC-init evidence에는 CandidateOrdinal, FreshSessionRetryReason, FreshSessionRetryDelayMs, FreshSessionRetryFromCandidate, FreshSessionRetryNextCandidate, FreshSessionFirstFailure가 포함된다.

같은 MainWindow와 fixed UDP port를 사용하는 Close -> Connect는 old endpoint 정리, 새 TCP session, callback endpoint 재등록을 모두 확인해야 한다. PC fake-RPC, loopback, process/mutex cleanup과 executable relaunch 시험은 이 계약의 PC 증거일 뿐 PLC callback disarm, PLC readiness, MotionLib 상태 또는 실제 장비 재접속을 증명하지 않는다. 배포 candidate는 복사 뒤 actual EXE gate와 tested/final binary identity 검사를 별도로 통과해야 한다. 최신 수치, artifact identity와 배포 상태는 API 개발 진척도에서 관리한다.

2.5 Safety transport preemption

일반 command가 transport를 점유하거나 결과 publication 대기 중일 때, 상위 safety owner가 local TCP transport를 폐기하고 새 session에서 safety command를 다시 소유하기 위한 PC-side API다.

```
public LMCsafetyPreemptionAbortEvidence
    AbortTransportForSafetyPreemption()

public LMCsafetyPreemptionAbortEvidence
    AbortTransportForSafetyPreemption(
        long expectedSessionGeneration)
```

이 호출은 RPC Close, Axis Stop, Group Stop 또는 Power Off frame을 보내지 않는다. local TCP client를 detach하고 evidence를 반환한다. 이후 reconnect, axis/group 재조회와 승인된 safety command 1회 전송이 필요하다. expectedSessionGeneration overload는 다른 session을 잘못 폐기하지 않기 위한 guard다.

LMCSendPriorityCoordinator는 ordinary/preemptible send와 priority send의 순서를 PC process 안에서 조정한다. 이 미 wire에 들어간 RPC를 물리적으로 취소하지 않으며 PLC E-stop/STO를 대체하지 않는다.

3. Single Axis API

3.1 LMCSingleAxis 생성

LASAL axis object name으로 axis reference를 가져온다.

```
public LMCSingleAxis(
    LMCCConnection connection,
    string axisName)

public static Task<LMCSingleAxis> CreateAsync(
    LMCCConnection connection,
    string axisName,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
connection	LMCCConnection	-	초기화된 RPC connection
axisName	string	ASCII string	PLC에 등록된 axis object name
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMCSingleAxis	조회된 axis 객체
Task<LMCSingleAxis>	비동기로 조회된 axis 객체

LMCAxis는 LMCSingleAxis의 호환 이름이다.

3.2 PowerOn

Axis Power On을 요청한다.

```
public LMC_Response PowerOn()
public Task<LMC_Response> PowerOnAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisPowerStateWaitResult>
    PowerOnAndWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisPowerStateWaitResult>
    ResumePowerOnWaitForStableStateAsync(
        LMCAxisPowerOnWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisPowerStateWaitResult> WaitForPowerStateAsync(
    bool expectedPowerOn,
    CancellationToken cancellationToken)

public LMCAxisPowerOnWaitContinuation PendingPowerOnWaitContinuation { get; }
public void ResolvePowerOnWaitAfterStablePowerOff(
    LMCAxisPowerOnWaitContinuation continuation)
```

Parameter	Type	UNIT	설명
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Power On command 결과

Return	설명
Task<LMC_Response>	비동기 Power On command 결과
Task<LMCAxisPowerStateWaitResult>	Power On ACK와 안정 상태 확인 evidence

PowerOnAndWaitForStableStateAsync는 0x2023을 한 번만 보내고 성공 ACK 뒤 0x2028의 PowerOn=true를 기본 3회 연속 확인한다. timeout/cancel 뒤에는 예외나 accepted observer가 보존한 LMCAxisPowerOnWaitContinuation을 ResumePowerOnWaitForStableStateAsync에 전달한다. Resume과 restart용 WaitForPowerStateAsync는 0x2028만 보내며 Power On을 다시 보내지 않는다. options overload로 total deadline, poll interval과 stable sample count를 설정할 수 있다. post-write deadline은 connection을 Faulted로 만들 수 있으므로 Evidence.SubmissionOutcome, CommandMayHaveBeenSent, PowerOnAccepted, TransportInvalidatedAtDeadline을 함께 확인한다. Power On 완료 조건은 read 성공과 PowerOn=true이며 axis error가 0일 필요는 없다. 따라서 성공 결과에서도 FinalStatus.HasAxisError, DS402 상태와 drive error를 별도로 확인한다.

3.3 PowerOff

Axis Power Off를 요청한다.

```
public LMC_Response PowerOff()
public Task<LMC_Response> PowerOffAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitContinuation>
    BeginPowerOffWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitResult>
    ResumePowerOffWaitForStableStateAsync(
        LMCAxisPowerOffWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisPowerOffWaitResult>
    PowerOffAndWaitForStableStateAsync(
        CancellationToken cancellationToken)

public LMCAxisPowerOffWaitContinuation PendingPowerOffWaitContinuation { get; }
```

Return	설명
LMC_Response	Power Off command 결과
Task<LMC_Response>	비동기 Power Off command 결과
Task<LMCAxisPowerOffWaitContinuation>	한 번 accept된 Power Off의 status-only 재개 정보
Task<LMCAxisPowerOffWaitResult>	PowerOn=false && Standstill=true 안정 evidence

Begin은 Power Off 0x2023을 한 번 보내고 ACK, process-local mutation generation과 accepted continuation을 session/send-priority publication 안에서 함께 저장한다. Resume은 0x2028만 polling하고 pre-wire/status publication/final resolution에서 원 generation을 확인한다. later same-axis mutation은 LMCAxisPowerOffInterferenceException과 expected/observed/intervening evidence를 반환하고 pending을 유지하며 PowerOff를 replay하지 않는다. final proof 전에 관찰된 cancel/deadline/generation change는 pending을 보존하고 proof commit 뒤 late cancel/deadline은 성공을 뒤집지 않는다. compound API는 두 단계를 같은 total deadline으로 조합한다. 외부 PLC/client/direct SDO/group은 이 귀속 범위 밖이며 성공 ACK만으로 전원 차단 완료를 판정하지 않는다.

3.4 Reset

Axis error reset을 요청한다.

```

public LMC_Response Reset()
public Task<LMC_Response> ResetAsync(
    CancellationToken cancellationToken)

public Task<LMCAxisResetWaitResult>
    ResetAndWaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisResetWaitContinuation>
    BeginResetAndWaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public Task<LMCAxisResetWaitResult>
    ResumeResetAndWaitForStableErrorClearanceAsync(
        LMC_AxisResetWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisStableErrorClearanceWaitResult>
    WaitForStableErrorClearanceAsync(
        CancellationToken cancellationToken)

public LMC_AxisResetWaitContinuation PendingResetWaitContinuation { get; }

```

Return	설명
LMC_Response	Reset command 결과
Task<LMC_Response>	비동기 Reset command 결과
Task<LMCAxisResetWaitContinuation>	한 번 accept된 Reset의 status-only 재개 정보
Task<LMCAxisResetWaitResult>	Reset ACK와 native axis error 0 안정 evidence

Begin은 0x2024를 한 번만 보내고 accepted continuation을 반환하며 status를 읽지 않는다. Resume은 0x2028만 보내 AxisErrorId == 0을 기본 3회 연속 확인한다. compound API는 두 단계를 한 total elapsed deadline으로 조합한다. timeout/cancel/status 실패 뒤에도 continuation을 status-only로 재개하고 Reset을 replay하지 않는다. invalid/stale/superseded/completed continuation과 concurrent second Resume은 zero-wire다. WaitForStableErrorClearanceAsync는 Reset을 보내지 않는 read-only helper라서 restart 뒤 상태 재확인에 사용할 수 있다.

같은 connection session과 AxisReference의 later LMCSingleAxis mutation이 may-have-been-sent boundary에 도달 하면 LMC_AxisResetInterferenceException으로 원 Reset 귀속을 거부한다. continuation은 pending으로 남지만 의도적인 post-Reset Power On을 포함해 간섭이 확인된 뒤에는 명시적인 새 Reset이 필요하다. 외부 PLC, 다른 RPC client, direct SDO와 group operation은 이 process-local 귀속 범위 밖이다. 현재 reserved StatusWord를 DS402 Fault 해제 증거로 해석하지 말고, 실제 장비에서는 DS402 Fault, AxError와 drive error register를 별도로 확인한다.

3.5 Stop

현재 axis motion의 정지를 요청한다.

```

public LMC_Response Stop(
    int deceleration,
    int jerk)

public Task<LMC_Response> StopAsync(
    int deceleration,
    int jerk,
    CancellationToken cancellationToken)

public Task<LMCAxisStopWaitContinuation>
    BeginStopWaitForStableStandstillAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    ResumeStopWaitForStableStandstillAsync(
        LMCAxisStopWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    StopAndWaitForStableStandstillAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCAxisStableStandstillWaitResult>

    WaitForStableStandstillAsync(
        CancellationToken cancellationToken)

public LMCAxisStopWaitContinuation PendingStopWaitContinuation { get; }

public Task<LMCAxisStopWaitContinuation>
    BeginStopWaitForStableStandstillWithResetTakeoverAsync(
        LMCAxisResetWaitContinuation resetContinuation,
        int deceleration,
        int jerk,
        LMCAxisStopWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCAxisStopWaitResult>
    StopAndWaitForStableStandstillWithResetTakeoverAsync(
        LMCAxisResetWaitContinuation resetContinuation,
        int deceleration,
        int jerk,
        LMCAxisStopWaitOptions options,
        Action<LMCAxisStopWaitContinuation> acceptedContinuationObserver,
        CancellationToken cancellationToken)

```

Parameter	Type	UNIT	설명
deceleration	int	PLC application UNIT/s ² DINT	양수인 정지 감속도. 0/음수는 송신 전에 거부
jerk	int	PLC application UNIT/s ³ /1000 DINT	0 이상인 정지 jerk. 음수는 송신 전에 거부
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Stop command 결과
Task<LMC_Response>	비동기 Stop command 결과
Task<LMCAxisStopWaitContinuation>	한 번 accept된 Stop의 status-only 재개 정보
Task<LMCAxisStopWaitResult>	안정 Standstill 확인 evidence

성공 응답은 LASAL `_LMCAxis.StopMove`가 오류 flag 없이 접수됐다는 뜻이다. 실제 정지 완료는 `ReadStatusResult[Async]`의 `IsStandstill`을 후속 확인해야 한다. 이 로컬 DINT 계약을 PMAS/MMCLib `MMC_Stop`의 function-block 완료 의미와 동일하게 해석하지 않는다. Begin은 0x2022를 정확히 한 번 보내고 continuation을 반환한다. Resume은 0x2028만 polling하며 기본 3회 연속 `IsSuccess` && `IsStandstill`을 확인한다. compound API는 두 단

계를 같은 total deadline으로 조합한다. timeout/cancel/status 실패 뒤 accepted continuation을 재개할 때 Stop을 다시 보내지 않는다. 같은 session/AxisReference의 later `LMCSingleAxis` mutation이 확인되면 `LMCAxisStopInterferenceException`을 반환하고 pending continuation을 보존한다. zero-wire mutation과 다른 AxisReference는 간섭하지 않으며 외부 PLC/client/SDO/group operation은 process-local 귀속 범위 밖이다. `WaitForStableStandstillAsync`는 Stop을 보내지 않고 0x2028만 읽는다. 이미 안정 Power Off가 증명된 뒤에는 `TryRetirePendingStopAfterStablePowerOff`로 정확한 pending Stop을 wire traffic 없이 superseded 처리할 수 있다. . . . `.WithResetTakeoverAsync` overload는 정확한 same-session pending Reset의 소유권을 새 Stop에 원자적으로 넘기는 고급 recovery API다. 임의의 stale/foreign Reset continuation에는 사용할 수 없으며 Stop ACK가 불명확하면 Reset이나 Stop을 blind retry하지 않는다.

3.6 ReadStatus

Axis 상태를 읽는다.

```
public uint ReadStatus()
public uint ReadStatus(out LMC_Response response)

public LMCReadStatusResult ReadStatusResult()
public Task<LMCReadStatusResult> ReadStatusResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
uint	Raw axis state
LMCReadStatusResult	Axis 상태와 error 정보
Task<LMCReadStatusResult>	비동기 axis 상태와 error 정보

3.7 GetActualPosition

Axis actual position을 읽는다.

```
public int GetActualPosition()
public int GetActualPosition(out LMC_Response response)

public LMCReadActualPositionResult GetActualPositionResult()
public Task<LMCReadActualPositionResult> GetActualPositionResultAsync(
    CancellationToken cancellationToken)
```

Return	UNIT	설명
int	PLC application UNIT DINT	Raw actual position
LMCReadActualPositionResult	PLC application UNIT DINT	Actual position과 error 정보
Task<LMCReadActualPositionResult>	PLC application UNIT DINT	비동기 actual position 결과

3.8 MoveAbsoluteEx

Axis를 absolute position으로 이동시킨다.

```

public LMC_Response MoveAbsoluteEx(
    int position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction = LMC_DIRECTION.Shortest)

public Task<LMC_Response> MoveAbsoluteExAsync(
    int position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT	설명
position	int	PLC application UNIT DINT	Absolute target position
velocity	int	PLC application UNIT/s DINT	Velocity
acceleration	int	PLC application UNIT/s ² DINT	Acceleration
deceleration	int	PLC application UNIT/s ² DINT	Deceleration
jerk	int	PLC application UNIT/s ³ /1000 DINT	Jerk
direction	LMC_DIRECTION	Shortest	이동 방향
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveAbsolute command 결과
Task<LMC_Response>	비동기 MoveAbsolute command 결과

3.9 MoveRelativeEx

현재 위치에서 지정한 distance만큼 이동시킨다.

```

public LMC_Response MoveRelativeEx(
    int distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction = LMC_DIRECTION.Shortest)

public Task<LMC_Response> MoveRelativeExAsync(
    int distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT	설명
distance	int	PLC application UNIT DINT	Signed relative distance
velocity	int	PLC application UNIT/s DINT	Velocity
acceleration	int	PLC application UNIT/s ² DINT	Acceleration
deceleration	int	PLC application UNIT/s ² DINT	Deceleration
jerk	int	PLC application UNIT/s ³ /1000 DINT	Jerk

Parameter	Type	UNIT	설명
direction	LMC_DIRECTION	Shortest	Distance 부호에 따른 이동
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveRelative command 결과
Task<LMC_Response>	비동기 MoveRelative command 결과

3.10 MoveVelocityEx

지정한 방향과 속도로 axis를 구동한다.

```
public LMC_Response MoveVelocityEx(
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction)

public Task<LMC_Response> MoveVelocityExAsync(
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMC_DIRECTION direction,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
velocity	int	PLC application UNIT/s DINT	Velocity magnitude
acceleration	int	PLC application UNIT/s² DINT	Acceleration
deceleration	int	0	0을 전달하고 정지는 Stop 사용
jerk	int	PLC application UNIT/s³/1000 DINT	Jerk
direction	LMC_DIRECTION	Positive, Negative	이동 방향
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveVelocity command 결과
Task<LMC_Response>	비동기 MoveVelocity command 결과

3.11 Drive operation mode와 composite status

physical axis/slave 1..4의 CiA 402 operation mode와 status를 D5 SDO Read로 조회한다.

```

public LMCDriveOperationModeResult GetDriveOperationMode()
public LMCDriveOperationModeResult GetDriveOperationMode(uint timeoutCycles)
public Task<LMCDriveOperationModeResult> GetDriveOperationModeAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveOperationModeResult> GetDriveOperationModeAsync(
    uint timeoutCycles,
    CancellationToken cancellationToken)

public LMCDriveErrorCodeResult GetDriveErrorCode()
public LMCDriveErrorCodeResult GetDriveErrorCode(uint timeoutCycles)
public Task<LMCDriveErrorCodeResult> GetDriveErrorCodeAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveErrorCodeResult> GetDriveErrorCodeAsync(
    uint timeoutCycles,
    CancellationToken cancellationToken)

public LMCDriveStatus ReadDriveStatus()
public LMCDriveStatus ReadDriveStatus(uint timeoutCycles)
public Task<LMCDriveStatus> ReadDriveStatusAsync(
    CancellationToken cancellationToken)
public Task<LMCDriveStatus> ReadDriveStatusAsync(
    uint timeoutCycles,
    CancellationToken cancellationToken)

```

GetDriveOperationMode는 0x6061:0 Int8/1을 읽어 typed Mode와 signed RawValue를 반환한다.

unknown/manufacturer-specific 값은 IsKnownMode=false여도 RawValue에 보존된다. GetDriveErrorCode는 0x603F:0 UInt16/2를 읽고 raw drive error code와 D5 ticket evidence를 반환한다. 값 0은 해당 read 시점의 drive error register 가 0이라는 뜻이며 이전 fault 이력이나 DS402 Warning 부재를 증명하지 않는다.

ReadDriveStatus는 LASAL ReadStatus, DS402 0x6041:0 BitField16/2, 0x6061:0 Int8/1을 순차 실행한다. 같은 EtherCAT cycle의 atomic snapshot이 아니므로 IsAtomicSnapshot은 항상 false다. AxisStatus, Ds402StatusWord, OperationModeResult와 software/hardware/DS402 limit indication을 source별로 확인한다.

timeoutCycles는 각 PLC SDO operation timeout이며 기본값은 1000 cycles다. library의 terminal status poll 간격은 capability의 BaseCycleTimeUs에서 계산하며 최대 poll 수는 timeoutCycles+32다. BaseCycleTimeUs=0이면 요청 전에 실패한다. terminal 실패는 LMCSdoReadOperationException, PC poll 한계는 LMCSdoReadPollingTimeoutException으로 ticket/status를 보존한다. 제출 뒤 async cancellation은 ticket을 포함한 LMCSdoReadWaitCanceledException을 발생시키고 PC wait만 중단하며, 이미 제출한 PLC ticket을 자동 cancel하지 않는다. 이미 진행 중인 status RPC는 응답을 끝까지 수신한 뒤 취소를 보고하므로 connection은 유지되고 보존된 ticket을 다시 조회할 수 있다.

3.11.1 SetOperationMode 개발 SDK

2.4-development SDK에는 CSP mode 8 복구 전용 SetOperationMode lifecycle이 들어 있다. 현재 PLC에는 0x7D23/0x7D24/0x7D25 route/handler가 없고 Admin capability bits 8/9/10도 OFF이므로 지원 API로 실행할 수 없다. PrepareSetOperationMode는 fresh capability triad와 ErrorCatalogVersion 6을 요구하며, 조건이 없으면 Start는 wire를 보내기 전에 실패한다.

단계	LMCSingleAxis API	Command	current PLC
Prepare	PrepareSetOperationMode	wire 없음	PC validation only
Start once	SetOperationMode[Async]	0x7D23	route 없음, zero-wire 차단
Exact outcome query	ReadSetOperationModeOutcome[Async]	0x7D24	route 없음
Exact terminal retirement	RetireSetOperationModeOutcome[Async]	0x7D25	route 없음

schema v1은 physical axis 1..4, Immediate-only, requested mode 8만 허용한다. Start ACK는 terminal이 아니므로 prepared command는 one-shot이다. TCP write 결과가 불확실해도 Start를 자동 재전송하지 않는다. recovery는 original key로 outcome만 읽고 exact terminal generation만 retire한다. Mode 1/3/6/7, arbitrary 0x6060 write와

HomeDS402Ex는 이 API 범위가 아니다.

3.12 Move 완료와 restart recovery 경계

MoveAbsoluteEx, MoveRelativeEx, MoveVelocityEx의 sync/async 메서드는 command ACK만 반환한다. 현재 SDK에는 public Move...AndWait, BeginMove..., ResumeMove... continuation API가 없다. MoveVelocityEx는 명시적 Stop 전까지 목표 완료점 자체가 없다.

위치 이동 완료가 필요하면 application이 ReadStatusResult[Async]와 GetActualPositionResult[Async]를 반복 조회해 Standstill, axis error와 목표 위치 tolerance를 함께 확인해야 한다. 예제 WPF의 stable-sample 감시와 journal은 application-local 정책이지 SDK의 durable Move completion token이 아니다.

timeout, disconnect 또는 process restart로 Move 결과가 불명확하면 같은 Move를 자동 재전송하지 않는다. 새 connection/object lookup 뒤 현재 status/position을 읽고, 필요한 경우 승인된 StopAndWaitForStableStandstillAsync 또는 Power Off를 정확히 한 번 수행한 후 다음 동작을 결정한다.

4. Group API

4.1 LMCGroupAxis 생성

LASAL group object name으로 group reference를 가져온다.

```
public LMCGroupAxis(
    LMConnection connection,
    string groupName)

public static Task<LMCGroupAxis> CreateAsync(
    LMConnection connection,
    string groupName,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
connection	LMConnection	-	초기화된 RPC connection
groupName	string	ASCII string	PLC에 등록된 group object name
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMCGroupAxis	조회된 group 객체
Task<LMCGroupAxis>	비동기로 조회된 group 객체

LMCGroup은 LMCGroupAxis의 호환 이름이다.

Group Power/Enable/Reset/Stop/Move의 성공 ACK는 method 호출 접수 결과다. 완료 상태는 GroupReadStatusResult와 필요 시 GroupReadActualPosition으로 확인한다. 현재 SetKin/Lock/Move는 X/Y/Z/U 축 1~4에만 적용된다. 이것은 9축 동시 group interpolation API가 아니다.

4.2 GetGroupMembersInfo

Group에 연결된 axis member 정보를 읽는다.

```
public LMC_Response GetGroupMembersInfo()

public LMCGroupMembersInfoResult GetGroupMembersInfoResult()
public Task<LMCGroupMembersInfoResult> GetGroupMembersInfoResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
LMC_Response	Member 조회 command 결과
LMCGroupMembersInfoResult	Axis count, reference, device ID와 axis name
Task<LMCGroupMembersInfoResult>	비동기 member 정보 결과

4.3 GroupPowerOn

Group member axis의 Power On을 요청한다.

```
public LMC_Response GroupPowerOn()
public Task<LMC_Response> GroupPowerOnAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitContinuation>
    BeginGroupPowerOnWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
    ResumeGroupPowerStateWaitForStableStateAsync(
        LMCGroupPowerStateWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
    GroupPowerOnAndWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult> WaitForPowerStateAsync(
    bool expectedPowerOn,
    CancellationToken cancellationToken)
```

Return	설명
LMC_Response	Group Power On command 결과
Task<LMC_Response>	비동기 Group Power On command 결과
Task<LMCGroupPowerStateWaitContinuation>	한 번 accept된 Group Power On의 status-only 재개 정보
Task<LMCGroupPowerStateWaitResult>	안정된 PowerOn=true 확인 evidence

Begin/compound API는 0x204A를 한 번 보내고 Resume은 0x2045만 polling한다. 기본 완료 조건은 read 성공과 PowerOn=true 3회 연속이다. WaitForPowerStateAsync는 Power command를 보내지 않는 read-only helper다. PendingGroupPowerStateWaitContinuation은 같은 connection/session/group의 latest unresolved Power command를 제공한다.

4.4 GroupPowerOff

Group member axis의 Power Off를 요청한다.

```
public LMC_Response GroupPowerOff()
public Task<LMC_Response> GroupPowerOffAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitContinuation>
    BeginGroupPowerOffWaitForStableStateAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupPowerStateWaitResult>
    GroupPowerOffAndWaitForStableStateAsync(
        CancellationToken cancellationToken)
```

Return	설명
LMC_Response	Group Power Off command 결과
Task<LMC_Response>	비동기 Group Power Off command 결과

Return	설명
<code>Task<LMCGroupPowerStateWaitContinuation></code>	한 번 accept된 Group Power Off의 status-only 재개 정보
<code>Task<LMCGroupPowerStateWaitResult></code>	안정된 <code>PowerOn=false</code> 확인 evidence

Begin/compound API는 `0x204B`를 한 번 보내고, 공통 `ResumeGroupPowerStateWaitForStableStateAsync`는 `0x2045`만 polling한다. 완료 조건은 read 성공과 `PowerOn=false` 3회 연속이다. Group Power wait는 member별 drive-ready나 DS402 fault 부재를 증명하지 않으므로 필요한 경우 개별 axis drive status를 추가 확인한다.

4.5 GroupEnable

Group motion profile을 lock한다.

```

public LMC_Response GroupEnable()
public Task<LMC_Response> GroupEnableAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitResult>
    GroupEnableAndWaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitContinuation>
    BeginGroupEnableAndWaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupEnableWaitResult>
    ResumeGroupEnableAndWaitForLockedStandbyAsync(
        LMCGroupEnableWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupLockedStandbyWaitResult>
    WaitForLockedStandbyAsync(
        CancellationToken cancellationToken)

public LMCGroupEnableWaitContinuation PendingGroupEnableWaitContinuation { get; }

public bool TryReleasePendingGroupEnableForRetry(
    LMCGroupEnableWaitContinuation continuation)
public bool InvalidatePendingGroupEnableWaitStatusProof()

```

Return	설명
<code>LMC_Response</code>	Profile lock command 결과
<code>Task<LMC_Response></code>	비동기 profile lock 결과
<code>Task<LMCGroupEnableWaitContinuation></code>	한 번 accept된 Group Enable의 status-only 재개 정보
<code>Task<LMCGroupEnableWaitResult></code>	accepted ACK와 안정된 PowerOn + Locked Standby evidence
<code>Task<LMCGroupLockedStandbyWaitResult></code>	command 없이 안정된 PowerOn + Locked Standby를 읽은 결과

stable wait는 mutation/status gate, fresh `0x2047`, 모든 `0x2045`와 poll delay에 하나의 total deadline을 적용한다. write commit 전 취소/deadline은 `NotAttempted`, zero wire이며 connection을 재사용하고 mutation proof는 변경하지 않는다. actual write commit의 `onWriteCommitted`에서만 mutation generation을 갱신하고 pending proof를 0으로 reset한다. caller cancel이 write 뒤 발생하면 ACK/status를 drain하고 accepted evidence를 게시한 뒤 typed cancellation을 반환한다. ACK 무응답 deadline은 `OutcomeUncertain`, continuation 없음, connection `Faulted`이고, ACK 수락 뒤 status 무응답은 `Accepted`, exact pending continuation, connection `Faulted`다. 두 경우 모두 `TransportInvalidatedAtDeadline=true`다. rejected ACK는 `Rejected`이고 continuation이 없다. 사용 가능한 동일 session의 accepted continuation은 Resume으로 `0x2045`만 poll하며 `0x2047`을 replay하지 않는다. options overload로 deadline, poll interval과 stable sample 수를 지정할 수 있다. 이 완료 판정은 PC fake-RPC 계약이며 실제 PLC profile lock 증거가 아니다. `TryReleasePendingGroupEnableForRetry`는 같은 group/session에서 Disabled/Unlocked 또는 PowerOff를 3회 안정 확인한 exact continuation만 wire 없이 해제한다.

`InvalidatePendingGroupEnableWaitStatusProof` 는 Stop/Power Off reservation 경계에서 누적 proof만 지우며

pending command를 완료시키지 않는다.

4.6 GroupDisable

Group motion profile을 unlock한다.

```
public LMC_Response GroupDisable()
public Task<LMC_Response> GroupDisableAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitContinuation>
    BeginGroupDisableWaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitResult>
    ResumeGroupDisableWaitForStableDisabledAsync(
        LMCGroupDisableWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupDisableWaitResult>
    GroupDisableAndWaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public Task<LMCGroupStableDisabledWaitResult>
    WaitForStableDisabledAsync(
        CancellationToken cancellationToken)

public LMCGroupDisableWaitContinuation PendingGroupDisableWaitContinuation { get; }
```

Return	설명
LMC_Response	Profile unlock command 결과
Task<LMC_Response>	비동기 profile unlock 결과
Task<LMCGroupDisableWaitContinuation>	한 번 accept된 Group Disable의 status-only 재개 정보
Task<LMCGroupDisableWaitResult>	안정된 powered-on Disabled 확인 evidence
Task<LMCGroupStableDisabledWaitResult>	command 없이 stable Disabled를 읽은 결과

Begin/compound API는 0x2048을 한 번 보내고 Resume은 0x2045만 polling한다. 기본 완료 조건은 read 성공, PowerOn=true, Disabled=true, Standby=false 3회 연속이다. 따라서 Power Off 상태를 Group Disable 완료로 오인하지 않는다. newer stable Group Power Off가 완료된 경우 TryRetirePendingGroupDisableAfterStablePowerOff는 정확한 pending Disable을 wire 없이 superseded 처리하지만, powered-on Disabled 완료를 주장하지 않는다.

4.7 GroupReset

Group error reset을 요청한다.

```

public LMC_Response GroupReset()
public Task<LMC_Response> GroupResetAsync(
    CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    BeginGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    BeginGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        Action<LMCGroupResetPreparedEvidence> preparedEvidenceObserver,
        Action<LMCGroupResetWaitContinuation> acceptedContinuationObserver,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitContinuation>
    AttachGroupResetDurableRecoveryAsync(
        LMCGroupResetDurableRecoveryRecord record,
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitResult>
    ResumeGroupResetWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitContinuation continuation,
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public Task<LMCGroupResetWaitResult>
    GroupResetAndWaitForStableErrorClearanceAsync(
        LMCGroupResetWaitOptions options,
        CancellationToken cancellationToken)

public bool SupersedePendingGroupResetAfterCapturedMemberSafetyMutation(
    LMCGroupResetWaitContinuation continuation,
    LMCSingleAxis memberAxis)

public LMCGroupResetWaitContinuation PendingGroupResetWaitContinuation { get; }

```

Return	설명
LMC_Response	Group Reset command 결과
Task<LMC_Response>	비동기 Group Reset command 결과
Task<LMCGroupResetWaitContinuation>	fresh Reset의 same-session 또는 exact durable reconnect/restart status-only 재개 정보
Task<LMCGroupResetWaitResult>	pinned group/member error-clear가 안정된 결과와 evidence

raw `GroupReset[Async]`는 ACK-only 호환 API다. stable Begin은 성공한 0x20D2 observed member snapshot을 고정한 뒤 0x2049를 정확히 한 번 보내고, Resume은 Reset을 재전송하지 않고 각 round에 0x2045 한 번과 pinned member별 0x2028을 보낸다. group/member error가 모두 0인 full-clear round를 기본 3회 연속 확인해야 완료다. generic snapshot validation은 1..16개 nonzero/unique axis reference를 허용하며 expected topology 또는 현재 PLC build 증명이 아니다.

timeout/cancel/status failure 뒤에는 같은 live session의 continuation으로 status-only Resume한다. accepted 또는 outcome-uncertain Stop/PowerOff/safe Disable이나 pinned-member mutation은 원 Reset 귀속을 terminal supersede하며, valid safety NACK와 pre-wire failure는 continuation을 보존한다. WPF 같은 coordinator가 captured-member Axis Stop/PowerOff 결과를 처리할 때는 `SupersedePendingGroupResetAfterCapturedMemberSafetyMutation`으로 exact continuation/member와 실제 generation mismatch를 확인해 SDK pending을 즉시 정리할 수 있다. valid NACK처럼 generation이 rollback된 경우 false를 반환하고 pending을 보존한다.

`preparedEvidenceObserver`는 valid 0x20D2 snapshot 뒤 실제 0x2049 write가 시작되기 직전에 operation ID, old owner session, exact ordered member identity와 stable count를 전달한다. caller는 이 동기 경계에서 command-before durable record를 먼저 저장해야 한다. observer가 실패하거나 reentrant mutation을 시도하면 0x2049는 전송되지 않는다.

process restart/reconnect에서는 old continuation을 재사용하지 않는다. endpoint, DiagnosticsBuild/BootId/MapRevision와 group identity를 먼저 확인한 뒤 `AttachGroupResetDurableRecoveryAsync`가 fresh `0x20D2`를 한 번 읽어 stored member count/order/name/ reference/device를 exact-match한다. 일치할 때만 새 session의 recovery continuation을 만들며 attach와 Resume 모두 `0x2049`를 보내지 않는다. stored prior outcome이 `OutcomeUncertain`이면 성공 결과도 현재 group/member error가 안정적으로 clear됐다는 사실만 뜻하며 이전 Reset ACK나 성공을 추론하지 않는다. WPF는 이 record를 checksum, single-writer lock과 exact CAS로 보존하고 stable proof 또는 accepted/outcome-uncertain safety supersede 뒤에만 resolve한다.

이 결과는 LASAL application error-clear 관찰이지 DS402 Fault, drive error register, power, profile lock, Home/reference 또는 motion-ready 증거가 아니다.

4.8 GroupStop

현재 group motion의 정지를 요청한다.

```
public LMC_Response GroupStop(
    int deceleration,
    int jerk)

public Task<LMC_Response> GroupStopAsync(
    int deceleration,
    int jerk,
    CancellationToken cancellationToken)

public Task<LMCGroupStopWaitContinuation>
    BeginGroupStopWaitForStableStandbyAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public Task<LMCGroupStopWaitResult>
    ResumeGroupStopWaitForStableStandbyAsync(
        LMCGroupStopWaitContinuation continuation,
        CancellationToken cancellationToken)

public Task<LMCGroupStopWaitResult>
    GroupStopAndWaitForStableStandbyAsync(
        int deceleration,
        int jerk,
        CancellationToken cancellationToken)

public LMCGroupStopWaitContinuation PendingGroupStopWaitContinuation { get; }
```

Parameter	Type	UNIT	설명
deceleration	int	Group application UNIT/s ² DINT	정지 감속도
jerk	int	Group application UNIT/s ³ /1000 DINT	정지 jerk
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Group Stop command 결과
Task<LMC_Response>	비동기 Group Stop command 결과
Task<LMCGroupStopWaitContinuation>	한 번 accept된 Group Stop의 status-only 재개 정보
Task<LMCGroupStopWaitResult>	안정 Standby 확인 evidence

deceleration/jerk 조합은 PLC 계약에 맞게 RPC 전에 검사한다. success ACK는 입력 검증, robot client 연결과 `StopMove(Mode:=3)` dispatch를 뜻하며 정지 완료가 아니다. `StopMove()` 반환 `StopCmdNo`는 오류 코드가 아니라 정지가 끝날 buffer command index다. 실제 완료와 profile error는 `GroupReadStatusResult`로 확인한다.

Begin은 0x2085를 정확히 한 번 보내고 success ACK를 connection/session/group/latest-pending에 묶인 continuation으로 반환하며 status를 읽지 않는다. Resume은 그 exact continuation으로 0x2045만 poll하여 IsStandby를 기본 3회 연속 확인한다. timeout/cancel/status failure 또는 send-priority preemption 뒤에도 accepted continuation과 evidence를 보존하며 owner session이 사용 가능하면 원 Stop 없이 Resume할 수 있다. TransportInvalidatedAtDeadline=true이면 그 session의 continuation은 재사용할 수 없고 reconnect 뒤에도 Stop을 자동 replay하지 않는다. PendingGroupStopWaitContinuation은 현재 handle과 같은 connection/session/group의 latest pending Stop만 노출한다. stale, superseded, completed continuation과 concurrent second Resume은 zero-wire로 거부된다. options overload로 deadline, poll interval과 stable sample 수를 지정할 수 있으며 compound API는 Begin과 Resume에 같은 elapsed total deadline을 적용한다.

4.9 GroupReadStatus

Group power와 profile 상태를 읽는다.

```
public uint GroupReadStatus()
public uint GroupReadStatus(out LMC_Response response)

public LMCGroupReadStatusResult GroupReadStatusResult()
public Task<LMCGroupReadStatusResult> GroupReadStatusResultAsync(
    CancellationToken cancellationToken)
```

Return	설명
uint	Raw group state
LMCGroupReadStatusResult	Group power, profile 상태와 error 정보
Task<LMCGroupReadStatusResult>	비동기 group 상태 결과

4.10 GroupReadActualPosition

Group actual position을 읽는다.

```
public LMCGroupReadActualPositionResult GroupReadActualPosition(
    LMC_COORD_SYSTEM coordinateSystem)

public Task<LMCGroupReadActualPositionResult> GroupReadActualPositionAsync(
    LMC_COORD_SYSTEM coordinateSystem,
    CancellationToken cancellationToken)
```

Parameter	Type	UNIT	설명
coordinateSystem	LMC_COORD_SYSTEM	Enum	None 또는 Acs member-slot alias
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	UNIT	설명
LMCGroupReadActualPositionResult	Group application UNIT DINT	slot 1..9 member position, slot 10..16 zero와 error 정보
Task<LMCGroupReadActualPositionResult>	Group application UNIT DINT	비동기 group position 결과

현재 adapter는 no-CalcModel static identity에서 None/Acs를 같은 GetRobotPosition(CoordSystem:=0) member-slot read alias로 처리한다. Mcs/Pcs는 지원하지 않아 C#에서 NotSupportedException이 발생하며, 구 SDK 요청은 PLC가 ErrorId=-7로 거부한다. slot 1..9는 software group member 순서이고 slot 10..16은 0이다. Acs alias의 실물 동등성은 PLC 시험이 남아 있다.

4.11 SetKinTransformCartesian4Axis

4개 axis를 Cartesian X/Y/Z/U로 설정한다.

```

public LMC_Response SetKinTransformCartesian4Axis(
    LMCSingleAxis axisX,
    LMCSingleAxis axisY,
    LMCSingleAxis axisZ,
    LMCSingleAxis axisU)

public Task<LMC_Response> SetKinTransformCartesian4AxisAsync(
    LMCSingleAxis axisX,
    LMCSingleAxis axisY,
    LMCSingleAxis axisZ,
    LMCSingleAxis axisU,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT	설명
axisX	LMCSingleAxis	-	Cartesian X axis
axisY	LMCSingleAxis	-	Cartesian Y axis
axisZ	LMCSingleAxis	-	Cartesian Z axis
axisU	LMCSingleAxis	-	Cartesian U axis
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	Kinematic transform command 결과
Task<LMC_Response>	비동기 transform command 결과

이 helper는 exact X/Y/Z/U identity payload만 만든다. generic/dynamic kinematic transform 계산이나 profile lock을 수행하지 않는다.

4.12 MoveLinearAbsoluteEx

Group을 Cartesian absolute position으로 linear 이동시킨다.

```

public LMC_Response MoveLinearAbsoluteEx(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk)

public LMC_Response MoveLinearAbsoluteEx(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options)

public Task<LMC_Response> MoveLinearAbsoluteExAsync(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    CancellationToken cancellationToken)

public Task<LMC_Response> MoveLinearAbsoluteExAsync(
    int[] position,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options,
    CancellationToken cancellationToken)

```

Parameter	Type	UNIT	설명
position	int[]	Group application UNIT DINT	wire 16개; 현재 PLC는 X/Y/Z/U slot 1..4만 사용하고 5..16은 0이어야 함
velocity	int	Group application UNIT/s DINT	Path velocity
acceleration	int	Group application UNIT/s ² DINT	Path acceleration
deceleration	int	Group application UNIT/s ² DINT	Path deceleration
jerk	int	Group application UNIT/s ³ /1000 DINT	Path jerk
options	LMCGroupMotionOptions	-	Coordinate, transition, buffer와 execute 설정
cancellationToken	CancellationToken	-	비동기 호출 취소 token

Return	설명
LMC_Response	MoveLinearAbsolute command 결과
Task<LMC_Response>	비동기 MoveLinearAbsolute command 결과

현재 C#과 PLC가 함께 허용하는 범위는 position slot 1..4 X/Y/Z/U, 나머지 0, 양수 velocity/acceleration/deceleration, 0 이상 jerk, coordinate `None`, transition `ExactStop/ContinuousDirect`, buffer `Aborting/Buffered`, `Execute=true`다. 정의됐지만 지원하지 않는 option은 RPC 전에 `NotSupportedException`으로 거부한다.

4.13 MoveLinearRelativeEx

Group profile의 마지막 buffered target을 기준으로 Cartesian relative distance를 원자적으로 적재한다. PC에서 현재 위치를 읽어 absolute target으로 변환하지 않는다.

```
public LMCAdminResponse MoveLinearRelativeEx(
    int[] distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk)

public LMCAdminResponse MoveLinearRelativeEx(
    int[] distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options)

public Task<LMCAdminResponse> MoveLinearRelativeExAsync(
    int[] distance,
    int velocity,
    int acceleration,
    int deceleration,
    int jerk,
    LMCGroupMotionOptions options,
    CancellationToken cancellationToken)
```

`distance`와 `dynamics/options`의 UNIT 및 허용 범위는 absolute move와 같다. 현재 PLC는 X/Y/Z/U slot 1..4만 사용하고 slot 5..16=0, coordinate `None`, transition `ExactStop/ContinuousDirect`, buffer `Aborting/Buffered`, `Execute=true`만 허용한다.

반환형은 `LMCAdminResponse`다. valid command rejection은 `LMCAdminCommandException.Response`에 Admin detail과 native error를 보존한다. success는 `MoveRelativeCoord`가 profile queue에 명령을 수락했다는 뜻이며 완료가 아니다. 이후 `GroupReadStatusResult[Async]`에서 InPosition/profile error를 확인한다.

일반 overload는 같은 session의 `0x7D00` capability를 먼저 확인한다. Stop/PowerOff 우선순위 gate가 있는 UI는 gate 밖에서 `GetCapabilitiesAsync`를 수행한 뒤 아래 prepared overload를 gate 안에서 사용한다. 전달한 capability는 같은 `LMCConnection`과 session, feature/group reference가 아니면 wire 송신 전에 거부된다.

```
LMCAdminCapabilities capabilities =
    await connection.Admin.GetCapabilitiesAsync(cancellationToken);

LMCAdminResponse accepted = await group.MoveLinearRelativeExAsync(
    distance,
    velocity,
    acceleration,
    deceleration,
    jerk,
    options,
    capabilities,
    cancellationToken);
```

prepared capability를 받는 sync overload도 같은 인자 순서로 제공한다.

4.14 LMCGroupMotionOptions

`MoveLinearAbsolute/Relative`의 좌표계와 motion mode를 설정한다.

Property	Type	UNIT / Default	설명
CoordinateSystem	LMC_COORD_SYSTEM	None	Coordinate system
TransitionMode	LMC_GROUP_TRANSITION_MODE	ExactStop	Transition mode
BufferMode	LMC_BUFFER_MODE	Aborting	Buffer mode
Execute	bool	true	Command execute

현재 PLC adapter가 승인하는 조합은 `CoordinateSystem=None, TransitionMode=ExactStop/ContinuousDirect, BufferMode=Aborting/Buffered, Execute=true`뿐이다. public enum에 다른 값이 있어도 현재 PLC 지원을 뜻하지 않는다.

5. Return Type

5.1 LMCReadStatusResult

`ReadStatusResult`의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
IsReadSuccessful	bool	-	RPC/function read 성공; native axis error 존재 여부와 분리
State	uint	Bit field	Raw axis state
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
IsPowerOn	bool	-	Axis Power On 상태
IsReferenced	bool	-	Home / Reference 완료 상태

Property	Type	UNIT	설명
IsStandstill	bool	-	Standstill 상태
AxisErrorId	ushort	Error ID	Axis error
HasAxisError	bool	-	<code>AxisErrorId != 0</code>
AxisErrorFlags	ushort	Bit field	raw LASAL <code>_LMCAXIS_ERROR</code> ; DS402 statusword bit가 아님
StatusWord	ushort	Reserved	현재 LASAL adapter는 0을 반환하며 DS402 statusword로 사용하지 않음
ErrorId	short	Error ID	Command error

5.2 LMCReadActualPositionResult

GetActualPositionResult의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
PositionRaw	int	PLC application UNIT DINT	Actual position
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
ErrorId	short	Error ID	Command error

5.3 LMCGroupReadStatusResult

GroupReadStatusResult의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
IsReadSuccessful	bool	-	RPC/function read 성공; native group error 존재 여부와 분리
State	uint	Bit field	Raw group state
IsPowerOn	bool	-	Group power 상태
IsStandby	bool	-	Profile locked 상태
IsEnabled	bool	-	<code>IsStandby</code> 호환 alias; servo power와 다름
IsDisabled	bool	-	Profile unlocked 상태
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
GroupErrorId	ushort	Error ID	Group / profile error
HasGroupError	bool	-	<code>GroupErrorId != 0</code>
ErrorId	short	Error ID	Command error

5.4 LMCGroupReadActualPositionResult

GroupReadActualPosition의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
CoordinateSystem	LMC_COORD_SYSTEM	Enum	요청에 사용한 좌표계의 PC-side echo; PLC 응답 필드가 아님
PositionsRaw	int[16]	Group application UNIT DINT	slot 1..9 software member position, slot 10..16 zero
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
ErrorId	short	Error ID	Command error

현재 tracked PLC source는 `_LMC_PROF_POS`의 Pos1..Pos9를 response slot 1..9에 복사하고 slot 10..16을 0으로 유지한다. Move/SetKin/Lock의 physical 4축 제한은 그대로다.

5.5 LMCGroupMembersInfoResult

GetGroupMembersInfoResult의 반환값이다.

Property	Type	UNIT	설명
Response	LMC_Response	-	원본 command response
IsSuccess	bool	-	전체 결과 성공 여부
AxisCount	byte	Count	Group member 수; 현재 tracked source는 9
Members	LMCGroupMemberInfo[]	-	Member 정보 배열
AxisReferences / DeviceIds / AxisNames	배열	-	방어 복사된 원본 member 배열
FunctionStatus	ushort	Bit field	MotionLib function status
HasCommandError	bool	-	FunctionStatus command-error bit 여부
ErrorId	short	Error ID	Command error

LMCGroupMemberInfo의 반환 필드는 다음과 같다.

Property	Type	UNIT	설명
Index	int	Index	0-based member index
AxisReference	ushort	Reference	Axis reference
DeviceId	ushort	Device ID	PLC device ID
AxisName	string	ASCII string	LASAL axis object name

5.6 LMCAdminResponse

LASAL-local Admin 명령의 공통 16-byte response를 보존한다.

Property	Type	설명
TransportResponse	LMC_Response	outer transport frame

Property	Type	설명
SchemaVersion / ResponseFlags	ushort	Admin schema와 reserved flags
CommandStatus	ushort	0 success, 1 domain rejection
ErrorId	short	Admin -31000, positive GroupProfile code 또는 adapter fallback -6
RequestId	uint	request echo
DetailCode / DetailCodeValue	enum / uint	typed/raw Admin detail
IsSuccess	bool	status/error/detail이 모두 success인지 여부

6. Admin과 Diagnostics facade

6.1 Admin capability와 semantic parameter read

연결된 `LMCConnection.Admin`에서 LASAL-local read-only API를 사용한다. 각 parameter read는 먼저 `0x7D00` capability와 허용 mask를 확인한다.

```
LMCAAdminCapabilities capabilities = connection.Admin.GetCapabilities();

LMCAxisParameterResult axisParameter = connection.Admin.ReadAxisParameter(
    axis,
    LMCAxisParameterKey.MaxVelocity);

LMCGroupParametersResult groupParameters = connection.Admin.ReadGroupParameters(
    group,
    LMCGroupParameterSelection.All);
```

async 메서드는 `GetCapabilitiesAsync`, `ReadAxisParameterAsync`, `ReadGroupParametersAsync`이며 `CancellationToken`을 받는다. 축/group 객체 overload 외에 `ushort` reference overload도 있다. 다른 connection 또는 reconnect 전 stale 객체는 거부한다.

axis read 제한:

- physical AxisReference 1..4
- key: `SoftwareMinPosition`, `SoftwareMaxPosition`,
`EndPositionToleranceWindow`, `MaxVelocity`, `MaxAcceleration`, `ReferencePosition`

- 한 호출에 한 key, 반환 value type `Int32`

`EndPositionToleranceWindow`는 profile의 in-position 상태가 아니라 축 end-position tolerance parameter다. 결과의 `Unit`과 `Value`를 함께 확인한다.

group read 제한:

- GroupReference `0x0100`
- selection: `PathVelocityLimit`, `PathAccelerationLimit`, `JerkTime` 또는 조합
- 최대 3개; unit은 각각 application UNIT/s, application UNIT/s2, milliseconds

지원하지 않는 PLC/schema/capability는 `LMCAAdminNotSupportedException` 또는 `NotSupportedException`, valid admin error response는 `LMCAAdminCommandException`으로 보고하며 `Response`에 status/error/detail을 보존한다. 이 API는 read-only이고 motion을 생성하지 않는다.

6.2 PI alias와 Bulk builder/reader

먼저 `GetSignalCatalog`로 얻은 같은 boot/map의 catalog를 사용한다.

```
LMCSignalValue value = connection.Diagnostics.ReadPI(
    catalog,
    "axis1.actual_position");

LMCPIBulkBuilder builder = connection.Diagnostics.CreatePIBulkBuilder(catalog);
builder.AddEntry("axis1.actual_position");
builder.AddEntry("axis2.actual_position");
LMCPIBulkReader reader = builder.Configure();
LMCBulkSnapshot snapshot = reader.Upload();
LMCSignalValueEntry entry = reader.GetEntry("axis1.actual_position");
reader.Release();
```

builder는 readable catalog entry, 중복, 최대 32개와 exact `MapRevision`을 검사한다. Configure 성공 뒤 builder는 frozen된다. `GetEntry/TryGetEntry`는 마지막 성공 Upload의 snapshot을 조회하며 새 PLC read를 수행하지 않는다. 별도 compatibility wire는 없고 D1 PI Read와 D2 Bulk command를 재사용한다. sync 메서드에는 대응하는 `ConfigureAsync`, `UploadAsync`, `ReadStatusAsync`, `ReleaseAsync`가 있다.

6.3 Project-local error catalog

```
LMCErrorDescription description;
if (LMCErrorCatalog.TryDescribe(
    LMCErrorDomain.AdapterCommand,
    response.ErrorId,
    out description))
{
    Console.WriteLine(description.Symbol);
    Console.WriteLine(description.Resolution);
}
```

지원 domain은 `AdapterCommand`, `AdminDetail`, `DiagnosticsDetail`, `GroupProfile`이다. 같은 숫자라도 domain마다 의미가 다르므로 domain을 추측하지 않는다. 반환 객체는 `Description`, `Resolution`, `CatalogVersion`, `SourceVersion`을 제공한다. unknown domain/value는 false를 반환한다. 이 catalog는 현재 project-local 계약이며 Elmo Maestro Personality 전체 error database가 아니다.

6.4 Diagnostics D0 capability와 공통 수명

Diagnostics는 `connection.Diagnostics`에서 시작한다. reconnect 뒤에는 capability를 다시 읽어야 하며, 이전 session에서 얻은 catalog, bulk configuration, recorder identity, operation ticket와 topology 객체를 재사용하지 않는다.

```
LMCDiagnosticCapabilities capabilities =
    connection.Diagnostics.GetCapabilities();

Task<LMCDiagnosticCapabilities> pending =
    connection.Diagnostics.GetCapabilitiesAsync(cancellationToken);
```

주요 반환값은 `BootId`, `MapRevision`, `Capabilities`, `BaseCycleTimeUs`, Catalog/Bulk/Recorder 최대 크기와 request/response payload 한도다. `BootId=0`은 초기 제한 상태이므로 Recorder나 SDO 같은 장기 작업을 시작하지 않는다. capability mask를 고정값으로 가정하지 말고 매 connection에서 다시 읽은 실제 반환값과 새 `BootId/MapRevision`을 사용한다.

기능 영역	현재 판정	증거 경계
Admin 0x7D00/10/20/22	source-active	전체 실물 값/UNIT/fault matrix는 별도 적격성 확인 필요
Catalog, PI Read, Bulk Read	capability-gated	typed catalog, revision과 partial/recovery를 함께 확인
D5 SDO Read	capability-gated	operation ticket terminal과 typed readback을 확인
Static Topology	source-active	configured inventory이며 runtime health 증거가 아님

기능 영역	현재 판정	증거 경계
EtherCAT Health	source-active	fault/stale/soak 실기 적격성은 별도 확인
Recorder Single/Ring/Trigger	source-active	capture 적격성과 storage 한계를 별도 확인
SDO Write	축 1 UI[24] exact target만 제한	fresh identity와 read-before/write/readback 절차가 필수; 축 2~4 차단
PI/DO Write, Recorder Double	현재 차단 또는 미구현	capability, route 또는 allowlist OFF
Node Health/DI	dormant source	capability OFF이며 정상 UI/API에서 호출 금지

ACK 또는 operation ticket은 작업 완료가 아니다. operation status의 terminal state와 result를 확인한다. `CancellationToken`은 PC 대기를 중단할 뿐 이미 PLC가 accept한 작업을 자동 취소하지 않는다. 결과가 불명인 mutation 또는 Release를 blind retry하지 않는다.

6.5 Diagnostics D1 Catalog, Health와 PI Read

```
LMCSignalCatalogInfo info = diagnostics.GetSignalCatalogInfo();
LMCSignalCatalogChunk chunk = diagnostics.GetSignalCatalogChunk(
    expectedMapRevision, startIndex, maxEntries);
LMCSignalCatalog catalog = diagnostics.GetSignalCatalog();

LMCEtherCATHealth health = diagnostics.ReadEtherCATHealth();
LMCSignalValue value = diagnostics.ReadPI(signalId);
LMCSignalValue typed = diagnostics.ReadPI(
    signalId, expectedMapRevision, expectedType);
```

각 메서드에는 `CancellationToken`을 받는 Async overload가 있다. `GetSignalCatalog()`는 Info와 chunk를 조합하고 count/CRC/map을 검사한다. 현재 catalog alias는 각 axis에 다음 6개다.

- `target_position_last_tx`
- `digital_outputs_last_tx`
- `control_word_last_tx`
- `actual_position`
- `digital_inputs`
- `status_word`

`ReadPI(catalog, alias)` facade는 catalog/session/map/type provenance를 wire 전 검사한다. 재접속 뒤 이전 catalog 나 alias lookup 결과를 사용하면 거부된다. 값은 raw DINT/application UNIT이며 library가 PMAS UNIT 변환을 수행하지 않는다.

`ReadEtherCATHealth`는 Master/Slave 상태, DS402 StatusWord와 Axis Error를 읽지만 drive-ready, DS402 Warning 부재, 물리 배선 또는 실제 I/O 정상 동작을 증명하지 않는다.

6.6 Diagnostics D2 Bulk Read

raw facade는 다음 순서로 사용한다.

```
LMCBulkConfiguration configuration = diagnostics.ConfigureBulk(signalIds);
LMCBulkStatus status = diagnostics.ReadBulkStatus(configuration);
LMCBulkSnapshot snapshot = diagnostics.ReadBulk(configuration);
diagnostics.ReleaseBulk(configuration);
```

각 메서드에는 Async overload가 있다. `ConfigureBulk` 뒤 `Active`를 확인하고, `ReadBulk`의 한 snapshot에서 같은 PLC cycle의 entry를 읽는다. `SnapshotFlags.Partial`이면 RPC 성공만으로 완료 처리하지 말고 각 entry의 `EntryStatus`와 detail을 검사한다. `ReleaseBulk` 결과가 불명확하면 새 session에서 자동 재전송하지 않는다.

일반 사용에는 6.2의 `LMCPIBulkBuilder/LMCPIBulkReader`를 권장한다. current 4-entry Pending -> Active -> Snapshot -> Release happy-path는 실기 확인됐지만, 최대 24-entry와 offline Partial/recovery/soak는 미완료다.

6.7 Diagnostics D3/D4 Recorder

대표 lifecycle API는 다음과 같다. 모두 session/BootId에 귀속되며 sync 메서드에는 해당 Async overload가 있다.

```
LMCRecorderConfigurationHandle handle =
    diagnostics.ConfigureRecorder(configuration);
LMCRecorderIdentity identity = diagnostics.StartRecorder(handle);

diagnostics.TriggerRecorder(identity);
diagnostics.StopRecorder(identity);
LMCRecorderStatus status = diagnostics.GetRecorderStatus(identity);
LMCRecorderHeader header = diagnostics.GetRecorderHeader(identity);
LMCRecorderChunk chunk = diagnostics.ReadRecorderChunk(chunkRequest);

LMCRecorderData data = await diagnostics.DownloadRecorderAsync(
    identity, progress, cancellationToken);
diagnostics.ReleaseRecorderBuffer(identity);
diagnostics.ReleaseRecorder(identity);
```

기본 순서는 Configure -> Start -> Status polling -> Ready -> Header/Chunk Download -> ReleaseBuffer -> Release다. `LMCRecorderData.GetRawUInt32/GetRawInt32`로 sample/channel raw 값을 읽을 수 있다. Single, Ring, Trigger mode는 source-active지만 current PLC runtime packet capture와 reconnect/fault 적격성은 완료되지 않았다.

`ReadRecorderBankInventory`, `ReadRecoverableRecorderBankInventory`, `AdoptRecorder`, `AdoptActiveRecorder` 등 recovery API도 공개되어 있다. 이들은 exact BootId/configuration/ identity와 journal proof를 요구하며 live reconnect/adopt matrix는 미완료다.

현재 사용 금지: `ConfigureRecoverableDoubleRecorder`, Double-bank inventory/adoption 계약은 공개되어 있지만 Recorder Double capability와 PLC route gate가 OFF이고 실제 bank count가 1이다. 현재 target에서는 `UnsupportedFeature` 대상이다.

6.8 Diagnostics D5 SDO Read와 operation ticket

```
LMCSdoReadResult inline = diagnostics.ReadSdoInline(request);
LMCOperationTicket ticket = diagnostics.SubmitSdo(request);
LMCOperationStatus status = diagnostics.GetOperationStatus(ticket);
diagnostics.CancelOperation(ticket);
LMCSdoResultChunk chunk = diagnostics.ReadSdoResultChunk(chunkRequest);
```

각 메서드에는 Async overload가 있다. current SDO Read 범위는 Slave 1.4, data width 1/2/4 byte, timeout 1.60000 PLC cycle과 general inline read다. `LMCSdoRequest.CreateRead`로 request를 만든다. 1/2/4-byte Read와 TypeMismatch 뒤 동일 BootId recovery happy-path는 실기 확인됐다.

`SubmitSdo`의 ticket을 받은 뒤 `GetOperationStatus`로 terminal Completed와 Success를 확인한다. `CancelOperation`도 operation 결과를 임의로 성공/실패로 확정하지 않으므로 status를 다시 읽는다. PC polling timeout/cancellation은 PLC operation을 자동 중단하지 않는다.

8/12-byte request/result chunk 계약은 공개되어 있으나 current maximum SDO data size가 4이고 Extended SDO capability/0x7E51 route가 OFF라 실행할 수 없다.

6.9 Static Topology와 Dynamic Node/I/O

```
LMCEtherCATTopologyInfo info = diagnostics.GetEtherCATTopologyInfo();
LMCEtherCATTopologyChunk chunk = diagnostics.GetEtherCATTopologyChunk(...);
LMCEtherCATTopology topology = diagnostics.GetEtherCATTopology();
```

Async overload도 제공한다. `GetEtherCATTopology()`는 Info 뒤 7개 chunk를 조합하고 CRC를 검증한다. current static inventory는 TopologyRevision `0x15867EEC`, 7 entry(Slave 5 + Slot Module 2)다. 이것은 프로젝트에 설정된 schema이며 실제 runtime node health, 물리 배선 순서, dynamic I/O 값 또는 drive-ready 상태를 증명하지 않는다.

다음 public contract가 존재한다. current LASAL source는 read-owner 두 command를 구현했지만 capability를 광고하지 않으므로 정상 public/WPF 경로에서는 preflight에서 차단된다. output write는 handler와 allowlist가 모두 없다.

API	Command	현재 상태
<code>ReadEtherCATNodeHealth[Async]</code>	<code>0x7E13</code>	LASAL handler/464-byte snapshot 구현, capability OFF, runtime proof 없음
<code>ReadDigitalIO[Async]</code>	<code>0x7E22</code>	LASAL handler/CREVIS input-output shadow 구현, capability OFF, runtime proof 없음
<code>SubmitDigitalOutputWrite[Async]</code>	<code>0x7E23</code>	capability OFF, RT owner/handler/allowlist 없음

`GetApprovedDigitalOutputWriteReferences()`는 현재 빈 목록이다. request DTO를 생성할 수 있다는 사실은 PLC 실행 지원을 뜻하지 않는다.

6.10 Mutation API 정책

Public API 계약	현재 차단 근거	판정
<code>SubmitPIWrite[Async]</code>	PI Write capability OFF, SDK/PLC allowlist 비어 있음, <code>0x7E21</code> unsupported	실행 금지
SDO CreateWrite + <code>SubmitSdo[Async]</code>	축 1, <code>0x2F00:24</code> , Int32, 4-byte만 SDK/PLC source 승인	조건부 제한 허용
<code>SubmitDigitalOutputWrite[Async]</code>	DO capability/route/owner/allowlist 없음	실행 금지
Recoverable Double Recorder	Double capability/route gate OFF, single bank	실행 금지

`GetApprovedSdoWriteTargets()`는 현재 축 1의 Gold UI[24] `0x2F00:24`, Int32/4-byte target 한 건만 반환한다. 축 2~4, 다른 index/subindex/type/length/value range 및 DS402 핵심 객체 `0x6040`, `0x607A`, `0x60FF`, `0x6071`은 SDK에서 wire 전에 차단된다.

규범적으로 유일한 SDO Write target은 Axis 1 `0x2F00:24`(UI[24]) Int32/4-byte다. Axis 2 through 4와 그 밖의 모든 target은 blocked 상태다. Manual SDO Write는 current-session identity-pinned 절차이며 `DiagnosticsBuild`, `BootId`, `MapRevision`과 exact target을 고정한 four-ticket same-value proof가 모두 PASS해야 한다.

`EvaluateSdoWritePolicy()`가 ready가 되려면 현재 connection에서 읽은 fresh capability의 `SDOWrite` bit 9, `SDORead`, `SDOReadGeneralInline`, nonzero `BootId/MapRevision`과 payload 한도가 모두 일치해야 한다. 기존 PLC가 bit 9를 광고하지 않으면 source 승인 target이 있어도 제출할 수 없다.

실제 Write 전에 축 1 drive program에서 UI[24]가 미사용임을 확인하고 PowerOff/Standstill, position 안정, 작업자 승인과 mutation journal을 모두 통과해야 한다. 최초 실기 시험은 baseline Read, 값 불변 pre-Write guard, byte-identical same-value Write, exact readback의 서로 다른 4개 ticket으로 제한한다. 이 4-ticket qualification이 PASS해야 일반 수동 Write 버튼이 열린다. 활성 proof는 exact `LMCConnection` 인스턴스, session generation, `DiagnosticsBuild`, `BootId`, `MapRevision`과 승인 target 전체 tuple에 묶이며 reconnect나 PLC identity 변경 시 폐기된다. identity mismatch나 disconnect를 한 번 관측하면 영구 폐기되며, 실제 second-click은 SDK가 mutation gate 안에서 fresh Build/BootId/MapRevision을 proof와 다시 비교한 뒤에만 `0x7E50`을 전송한다. mismatch는 `NotAttempted`이며 Write frame은 0회다. 결과가 불명확한 Write는 자동 재전송하지 않는다. current image load와 별개로 UI[24] 실제 소유권, 안전 절차와 EtherCAT mailbox mutation은 미검증이다.

6.11 Request/result와 provenance 확인

새 Diagnostics API는 request, options, continuation, ticket, identity와 result DTO를 분리한다. 다음 원칙을 공통 적용한다.

확인 항목	사용 규칙
IsSuccess, typed state/outcome	outer frame 성공과 domain operation 완료를 모두 확인
BootId, MapRevision, session generation	현재 connection에서 얻은 객체만 사용
BelongsTo / BelongsToCurrentSession	catalog, ticket, topology, I/O/result provenance 검증
RequestId, operation/configuration/recorder ID	ACK, status, chunk와 release 대상의 identity 일치 확인
Detail, ErrorId, failure context TryGet	예외에서 typed evidence를 보존하고 숫자만 추측하지 않음
Async CancellationToken	PC wait 취소이며 PLC mutation 취소 보장 아님

이 설명서는 principal facade와 안전 경계를 설명한다. 모든 DTO property, overload와 exception의 완전한 선언 목록은 배포 DLL의 IntelliSense/XML documentation과 current source를 함께 확인한다.

6.12 LMC Home, DS402 Home과 Encoder Maintenance

LMC Home CurrentPositionZero

LMCSingleAxis의 current LMC_Home은 switch-search reference가 아니다. 실행 전에 읽은 actual position을 ExpectedActualPosition stale-read guard로 사용하고 target position은 0으로 고정한다. axis motion을 enable하거나 Home/limit switch를 찾지 않는다.

단계	Public API	Command
Prepare	PrepareLMC_Home	wire 없음
Start once	LMC_Home, LMC_HomeAsync	0x7D13
Exact outcome query	ReadLMC_HomeOutcome, ReadLMC_HomeOutcomeAsync	0x7D18
Exact terminal retirement	RetireLMC_HomeOutcome, RetireLMC_HomeOutcomeAsync	0x7D19

```

LMCPreparedHome prepared = axis.PrepareLMC_Home(
    expectedActualPosition,
    timeoutMilliseconds,
    adminCapabilities,
    diagnosticCapabilities,
    LMCHomeExecuteToken.Create());

LMCHomeStartAcknowledgement accepted = axis.LMC_Home(prepared);
LMCHomeOutcomeResult outcome = axis.ReadLMC_HomeOutcome(
    accepted.RecoveryKey,
    adminCapabilities,
    diagnosticCapabilities);

if (outcome.IsTerminal)
{
    axis.RetireLMC_HomeOutcome(
        outcome,
        adminCapabilities,
        diagnosticCapabilities);
}

```

Start ACK는 완료 증거가 아니며 0x7D13을 replay하지 않는다. terminal outcome과 matching retirement가 필요하다. Admin feature bit 4는 current source에서 ON이고 WPF는 LMC Home outcome: 로그에 record state, success, original

status/error/detail, axis status/error, raw/application/internal position set, native/evidence/stop/runtime/generation 을 기록한다. 성공 raw feedback은 wrap-safe -2/-1/0/+1/+2 count 창으로 제한하고 +/-3 count 이상은 fail-closed한 다. raw before/after는 물리 feedback 증거이지 bit-identical sample 계약 이 아니다. Axis2의 8382700 -> 8382701과 Axis1의 8027834 -> 8027836을 이전 raw gate가 -7로 오판한 뒤 이 창을 PLC/SDK에 동기화했다. current image의 build/download와 별개로 새 BootId에 묶인 한 축 단독 terminal/physical proof는 기능 적격성 증거로 별도 확보해야 한다.

DS402 Home method 37

별도 LMC_HomeDS402는 method 37, Home offset 0, velocity/acceleration/distance/torque 0의 non-moving current-position-zero 계약이다.

단계	Public API	Command
Prepare	PrepareLMC_HomeDS402 또는 PrepareDs402Home	wire 없음
Start once	LMC_HomeDS402[Async] 또는 Ds402Home[Async]	0x7D15
Exact outcome query	ReadDs402HomeOutcome[Async]	0x7D16
Exact terminal retirement	RetireDs402HomeOutcome[Async]	0x7D17

current source에는 protocol/state-machine/API가 있지만 LMC_DIAG_DS402_HOME_ENABLED=FALSE이고 Admin feature bit 6도 OFF다. 따라서 current PLC의 지원 API로 실행하지 않는다.

TW[20] / TW[19] Encoder Maintenance

이 경로는 일반 SDO Write와 분리된 파괴적 유지보수 API다. 허용 payload는 TW[20] 0x20FC:0x02 <- UInt16 1, TW[19] 0x20FC:0x01 <- UInt16 1뿐이다.

단계	Public API	Command
Prepare TW[20]	PrepareTw20EncoderErrorWarningReset	wire 없음
Prepare TW[19]	PrepareTw19MultiturnPositionReset	wire 없음
Start once	StartEncoderMaintenance[Async]	0x7E53
Exact outcome query	ReadEncoderMaintenanceOutcome[Async]	0x7E54
Exact terminal retirement	RetireEncoderMaintenanceOutcome[Async]	0x7E55

각 Prepare는 대응 request와 one-shot execute token을 받는다: LMCTw20EncoderErrorWarningResetRequest + LMCTw20EncoderErrorWarningResetExecuteToken.Create(), 또는 LMCTw19MultiturnPositionResetRequest + LMCTw19MultiturnPositionResetExecuteToken.Create(). current source는 diagnostics capability bit 18/19를 ON으로 광고한다. 그러나 ACK나 terminal outcome은 정확한 drive error/warning reset 또는 multi-turn position 변화의 물리 증거가 아니므로 선택 측에서 별도 확인한다.

6.13 Axis SetPosition

SetPosition은 축의 application actual/destination position을 같은 target으로 교정하기 위한 project-local Admin 계약이다. 일반 motion 명령이 아니며, 현재 배포 image에서는 의도적으로 fail-closed다. 아래 API는 향후 durable Store와 native executor가 승인된 뒤 사용할 계약을 설명한다. 현재 장비에서 실행 절차로 사용하면 안 된다.

단계	Public API	Command	Frame
Prepare	PrepareSetPosition	wire 없음	-
Start once	SetPositionEx, SetPositionExAsync	0x7D12	request 56 B / response 36 B
Exact outcome query	ReadSetPositionOutcome, ReadSetPositionOutcomeAsync	0x7D14	request 60 B / terminal response 92 B
Exact terminal retirement	RetireSetPositionOutcome, RetireSetPositionOutcomeAsync	0x7D1A	request 64 B / terminal response 92 B

주요 signature는 다음과 같다.

```
LMCPreparedAxisSetPosition PrepareSetPosition(
    int targetPosition,
    int expectedActualPosition,
    LMCAdminCapabilities verifiedCapabilities,
    LMCDiagnosticCapabilities verifiedDiagnosticCapabilities,
    LMCAxisSetPositionExecuteToken executeToken)

LMCAxisSetPositionResult SetPositionEx(
    LMCPreparedAxisSetPosition preparedCommand)

LMCAxisSetPositionOutcomeResult ReadSetPositionOutcome(
    LMCAxisSetPositionRecoveryKey recoveryKey,
    LMCAdminCapabilities verifiedCapabilities,
    LMCDiagnosticCapabilities verifiedDiagnosticCapabilities)

LMCAxisSetPositionOutcomeRetirementResult RetireSetPositionOutcome(
    LMCAxisSetPositionRecoveryKey recoveryKey,
    uint recordGeneration,
    LMCAdminCapabilities verifiedCapabilities,
    LMCDiagnosticCapabilities verifiedDiagnosticCapabilities)
```

각 sync 메서드에는 `CancellationToken`을 받는 Async overload가 있다. retirement는 exact terminal `LMCAxisSetPositionOutcomeResult`를 받는 convenience overload도 제공한다.

Prepare는 current connection/session, axis, Diagnostics build/BootId/map revision, 128-bit client intent, target, expected actual position과 one-shot execute token을 고정한다. semantic mode는 `ActualAndDestinationApplicationUnits=1`뿐이며 position 값은 application UNIT DINT다. `expectedActualPosition`은 stale-read CAS다. prepared command는 TCP write boundary를 한 번만 통과할 수 있고, write 이후 결과가 불명확해도 0x7D12를 replay하지 않는다.

`LMCAxisSetPositionRecoveryKey`는 original request의 Diagnostics identity와 네 intent word, axis, target, expected actual position을 보존한다. outcome query는 이 original BootId와 별도로 현재 connection에서 새로 읽은 current BootId를 보내므로, PLC reboot 뒤에도 현재 PLC identity를 먼저 확인한 뒤 original record를 exact-match하도록 설계되어 있다. terminal outcome는 `Succeeded=2` 또는 `Rejected=3`, original status/error/detail, applied position, native state와 nonzero `RecordGeneration`을 반환한다. retirement는 exact key와 nonzero generation의 CAS이며, 성공 response가 유실돼도 같은 key/generation 재시도가 안전하려면 PLC가 durable tombstone을 보존해야 한다.

current image의 실제 경계는 다음과 같다.

- Admin capability bits 3/5/7이 OFF다. SDK capability preflight를 우회하지 않는다.
- Store와 ordinary ownership gate는 `FALSE`, 축 1~4 max-jump는 0이다.
- 1,344-byte backing은 ordinary volatile `VAR_GLOBAL`이며 restart/power-loss persistence가 없다.
- Admin SetPosition 경로의 native `.SetPosition()` call site는 0개다.
- exact raw start/query/retire 요청은 storage unavailable, detail 24로 fail-closed된다.

- 따라서 success, restart replay, outcome query와 retirement를 current 지원 기능으로 간주하지

않는다. WPF recovery journal도 이 경로로 해소할 수 없다.

활성화하려면 durable backend, boot/restart persistence, exact tombstone, axis task/core/priority, claim-before-native exactly-once 실행, terminal commit/readback, disconnect quarantine, packet capture와 실제 축 coordinate effect를 하나의 승인 evidence로 닫아야 한다.